Date.

编译程序是一种翻译程序，将源语言翻译为目标语言。

例：C语言代码 ⟶ 汇编代码

逻辑过程

词法分析 语法分析 语义分析 中间代码生成(优化) 目标代码生成(优化)

字母表 $\Sigma=\{a,b,c\}$ a, ab, aac 都是 Σ 上的符号串

空串 ε 长度 $|ab|=2$ $|\varepsilon|=0$

对于 $x=abc$，a、ab、abc 都是其子串 ba、ε 都不是

符号串的头尾、连接、幂 (P21)

符号串集合：若集合A中的一切元素都是某字母表 Σ 上的符号串，则称A为该字母表 Σ 上的符号串集合。

$\Sigma=\{1,2,3\}$ $A=\{12,13,23\}$

枚举表示法：$\{1,11,111\}$

省略表示法：$\{1,11,111,1111,\cdots\}$

描述表示法：$\{x \mid x$ 满足条件 $c\}$

$\Sigma=\{1\}$ $A=\{x \mid x$ 是该字母表上的符号串，且 $1\le|x|\le3\}$

空集：不含任何元素的集合，记为 $\emptyset$

两个符号串集合A与B的乘积 $AB=\{xy \mid x\in A$，且 $y\in B\}$

例如：$A=\{1,2\}$ $B=\{5,6,7\}$

则 $AB=\{15,16,17,25,26,27\}$

Σ 的闭包 $\Sigma^*=\Sigma^0\cup\Sigma^1\cup\Sigma^2\cup\cdots\cup\Sigma^n\cup\cdots$

Σ^* 表示 Σ 上一切长度为 n ($n\ge0$) 的符号串所组成的集合

Σ 的正闭包 $\Sigma^+=\Sigma^1\cup\Sigma^2\cup\cdots\cup\Sigma^n\cup\cdots$

显然，$\Sigma^*=\Sigma^0\cup\Sigma^+$，$\Sigma^+=\Sigma\Sigma^*=\Sigma^*\Sigma$

$\Sigma=\{a,b,c\}$ $\Sigma^*=\{\varepsilon,a,b,ab,ac,aa,ba,bb,bc,cc,ca,cb,c,\cdots\}$

一个文法 G[Z] 的 LR(0) 项集规范族的构造步骤：

1. 初始项集 S_0 = CLOSURE({Z'→·Z})，是 G 的 LR(0) 项集，这里 Z' 是包含有规则 Z'::=Z 的增广文法之识别符号
2. 如果 S_i 是 G 的项集，则 S_i 的一切后继项集均是 G 的项集
3. 重复步骤 2，直到再无新的项集可以添入。

特别说明：

I. 某一项集中，不同项，其后继符号相同时，后继项集相同

II. 不同项集中，若干个与前面对应相同的项，其后继项集与前面的相同

例：设有文法 G[S]：

(1) S::=A　　(4) A::=c

(2) S::=B　　(5) B::=aBb

(3) A::=aAb　　(6) B::=d

解：增广文法 G'[S']：(0) S'→S

(1) S→A　　(4) A→c

(2) S→B　　(5) B→aBb

(3) A→aAb　　(6) B→d

列表

状态	项目集	后继符号	后继状态
S_0	{S'→·S	S	S_1
	S→·A	A	S_2
	S→·B	B	S_3
	A→·aAb	a	S_4
	B→·aBb	a	S_4
	A→·c	c	S_5
	B→·d}	d	S_6

文法与语言

文法G[<标识符>]：（上下文无关文法）

<标识符> → <字母>　　<标识符> ::= <字母>

<标识符> → <标识符><数字>

<标识符> → <标识符><字母>

<字母> → a|b|c|…|z

<数字> → 0|1|2|…|9

重写规则（产生式）

元语言符号：→和::=表示"定义为"　|表示"或者"

左部　右部

文法G定义为四元组（V_N, V_T, P, S）

V_N 非终结符集　V_T 终结符集

P 重写规则集合　S 识别符号，$S \in V_N$

V_N = {<标识符>, <字母>, <数字>}

V_T = {a, b, c, …, z, 0, 1, 2, …, 9}

P = {<标识符> → <字母>,

<数字> → 0|1|2|…|9}

S = <标识符>

显然 $V_N \cap V_T = \emptyset$　$V = V_N \cup V_T$ 字汇表

(P22)

Date.

→ ::=

v = 233<字母>666　w = 233a666

<字母>::= a　则称 v 直接推导/直接产生出 w　记为 $v \Rightarrow w$

或称 w 直接归约到 v　记为 $w \Rightarrow v$

$v \Rightarrow w$。推导长度 $n \geq 1$

$v = w$。推导长度 $n = 0$

n步推导 $v \overset{+}{\Rightarrow} w$　$n \geq 1$

n次推导 $v \overset{*}{\Rightarrow} w$　$n \geq 0$ ↑包含 $v = v$

$G[Z]$ 字汇表 V　如果 $Z \overset{*}{\Rightarrow} x$，$x \in V^*$　称 x 为句型

如果 $Z \overset{*}{\Rightarrow} x$，$x \in V_T^*$　称 x 为句子

文法描述的语言是该文法一切句子的集合

$L(G[S]) = \{x \mid S \overset{*}{\Rightarrow} x$，且 $x \in V_T^*\}$

递归规则（右部出现左部）

文法 ⇄ 语言（唯一 / 不唯一）

若 $L(G_1) = L(G_2)$，则 等价

乔姆斯基 Chomsky 的语言分类法

0型文法　1型文法（上下文相关文法）

2型文法（上下文无关文法）

对于某文法 G，P 中的每个规则具有下列形式

$\alpha \to \beta$，其中 $\alpha \in V_N$，$\beta \in V^*$

3型文法（正规文法）

对于某文法 G，P 中的每个规则具有下列形式

$A ::= a$

或 $A ::= aB$（右线性文法）

$A ::= a$

或 $A ::= Ba$（左线性文法）

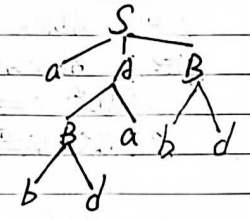
deli得力

Date.

求句型abdabd的短语.
简单短语.句柄

abdabd是句型abdabd相对于S的短语
bda是句型abdabd相对于A的短语
bd是句型abdabd相对于B的短语
bd是句型abdabd相对于B的简单短语
句柄: bd

语法分析树(推导树)



给有文法 G[S]: S→aAB
A→Ba|a
B→bd

S⇒aAB⇒aBaB⇒abdaB
⇒abdabd

每1结点都是某一终结符/非终结符
树根是文法的开始符
分枝
子树 简单子树(只含有单层分枝的子树)
(根据和其他无关当的部分)

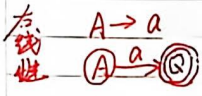
最左推导:每步直接推导总是替换最左的非终结符(最左边的)
最右推导(规范推导)
S⇒aAB⇒aAbd⇒aBabd⇒abdabd

归约序列: abdabd ⇐ aBabd ⇐ aAbd ⇐ aAB ⇐ S
规范句型(右句型) (最左规约)规范句约
二义性 (句子 文法 语言)
G[E]: E→E+E|E*E|(E)|i
推导句子 i*i+i (P28)

句型分析:自顶向下,自底向上(P30)
短语 简单短语 句柄(P31)
(直接短语)

DFA 状态转换
G[Z]: Z→Z0
A→Ba
B→Ab

左线性 A→a


右线性 A→a


五元组 DFA
其中

Date. /

能够能够的经过型abdabd概括于B的简单经过bd

正规式 ⟺ 正规文法

↓ ↓

正规集 语言(正规语言)

P4-

(a|b|…|z)(a|b|…|z|1|2|…|9|0)*

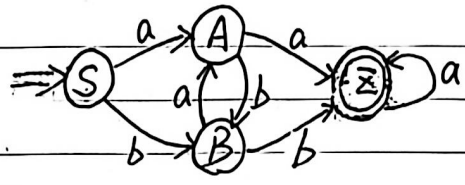
G[<标识符>]

DFA 状态转换图

G[Z]: Z→Za|Aa|Bb

A→Ba|a

B→Ab|b



左线性 A→a A→Ba…



例: ababaaa →

右线性 A→a A→aB



A→aB

A a→B

五元组 DFA $D_1=(K,\Sigma,M,S,F)$

其中 $K=\{S,A,B,Z\}$

$\Sigma=\{a,b\}$

M: $M(S,a)=A$ $M(S,b)=B$

$M(A,a)=Z$ $M(A,b)=B$

$M(B,a)=A$ $M(B,b)=Z$

$M(Z,a)=Z$

$S=S$

$F=\{Z\}$

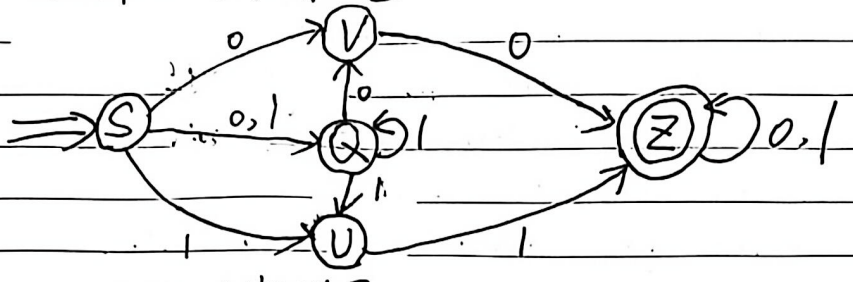
deli得力

Date. /

NFA的确定化　　子集法（可处理ε规则）ε-闭包

"简化的子集法"（不含ε规则）

例：将图中的NFA确定化



解：五元组　状态转换图

状态转换表（矩形表式）

K \ Σ	0	1
S	V,Q	Q,U
V	Z	
Q	V	U,Q
U		Z
Z	Z	Z

构造等价的DFA（K', Σ, M', S', F'）

M'：

K' \ Σ	0	1
[S]	[V,Q]	[Q,U]
[V,Q]	[V,Z]	[Q,U]
[Q,U]	[V]	[Q,U,Z]
[V,Z]	[Z]	[Z]
[V]	[Z]	
[Q,U,Z]	[V,Z]	[Q,U,Z]
[Z]	[Z]	[Z]

S': [S]

F': {[V,Z], [Q,U,Z], [Z]}

Date.

自顶向下的语法分析

LL(1)文法

First(α) 首符号集 $\alpha \in V^*$

即找最左边可能出现的终结符

例: G[E]: $E \to TE'$ $E' \to +TE' \mid \varepsilon$

$T \to FT'$ $T' \to *FT' \mid \varepsilon$

$F \to (E) \mid i$

First(E) = {(, i}

First(T) = {(, i}

First(F) = {(, i}

First(E') = {+, ε}

First(T') = {*, ε}

First(TE') = {(, i}

First(+TE') = {+}

First(FT') = {(, i}

First(*FT') = {*}

First((E)) = {(}

First(i) = {i}

看左边

Follow(A): 后跟符号集 $A \in V_N$ 看右边

即找出该非终结符后面所能跟随的所有终结符

1. # ∈ Follow(S) S为识别符号
2. 若存在规则 $V \to xWy$ First(y) - {ε} ∈ Follow(W)
3. 若存在规则 $V \to xW$ 或 $V \to xWy$, 其中 $y \overset{*}{\Rightarrow} \varepsilon$ 则 Follow(V) ∈ Follow(W)

Follow(E) = {#,)}

Follow(T) = {+, #,)}

Follow(E') = {#,)}

Follow(T') = {+, #,)}

Follow(F) = {*, +, #,)}

deli得力

Date. /

SELECT(A→α) 选择符号集

$\alpha \not\Rightarrow \varepsilon$ SELECT(A→α) = First(α)

$\alpha \Rightarrow \varepsilon$ SELECT(A→α) = (First(α) − {ε}) ∪ Follow(A)

SELECT(E'→+TE') = {+}
SELECT(E'→ε) = {), #}
SELECT(T'→*FT') = {*}
SELECT(T'→ε) = {+,), #}
SELECT(F→(E)) = {(}
SELECT(F→i) = {i}

LL(1)文法 (自顶向下分析技术)

对于每个非终结符A的两个产生式 A→α, A→β

满足: SELECT(A→α) ∩ SELECT(A→β) = ∅

(α和β至多有一个能推导出ε)

L abcd　L最左推导　| 向后看一个符号

判定: 含相同左部的产生式的可选集交集均为空集, 则该文法是LL(1)文法

非LL(1)文法到LL(1)文法的等价变换

一、提取左公共因子

一般形式 $A \to \alpha\beta_1 | \alpha\beta_2 | \alpha\beta_3 | \cdots | \alpha\beta_n$

提取 $A \to \alpha(\beta_1 | \beta_2 | \beta_3 | \cdots | \beta_n)$

引 $A \to \alpha A'$

$A' \to \beta_1 | \beta_2 | \cdots | \beta_n$

Date. /

例. G[S]: S:=AB
A:=aAb|ab
B:=cBd|de
提取左公共因子后的文法: G'[S]: S:=AB A:=aA'
A':=Ab|b B:=cBd|de

二、消除左递归
1. 消除直接左递归
对 $A\to A\alpha_1|A\alpha_2|\cdots|A\alpha_m|\beta_1|\beta_2|\cdots|\beta_n$
改写为: $A\to\beta_1A'|\beta_2A'\cdots|\beta_nA'$
$A'\to\alpha_1A'|\alpha_2A'\cdots|\alpha_mA'|\varepsilon$
这里, α_i 不等于 ε, β_j 不以A开头 A→A

例: G[E]: E→E+T|E-T|T
T→T*F|T/F|F
F→(E)|i
消除左递归后的文法: G'[E]: E→TE'
E'→+TE'|-TE'|ε
T→FT'
T'→*FT'|/FT'|ε
F→(E)|i

2. 消除间接左递归
3. 消除文法中的一切左递归

LL(1)分析的实现 { 递归下降
表驱动

deli得力

Date. /

递归下降分析技术是一种确定的自顶向下分析技术

基本思想：识别程序由一组子程序组成，其中每个子程序对应于文法的一个非终结符号；由于文法通常是递归定义的，因此这些子程序往往是递归子程序。

文法中每一个非终结符所对应的子程序，根据选定的产生式右部出现的符号来设计，具体构造方法如下：

1. 每遇到一个终结符，则判断当前输入单词是否与该终结符相匹配，若匹配，则继续读取下一个单词；若不匹配，则进行错误处理。

2. 每遇到一个非终结符，则调用相应的子程序。

文法G[E]：

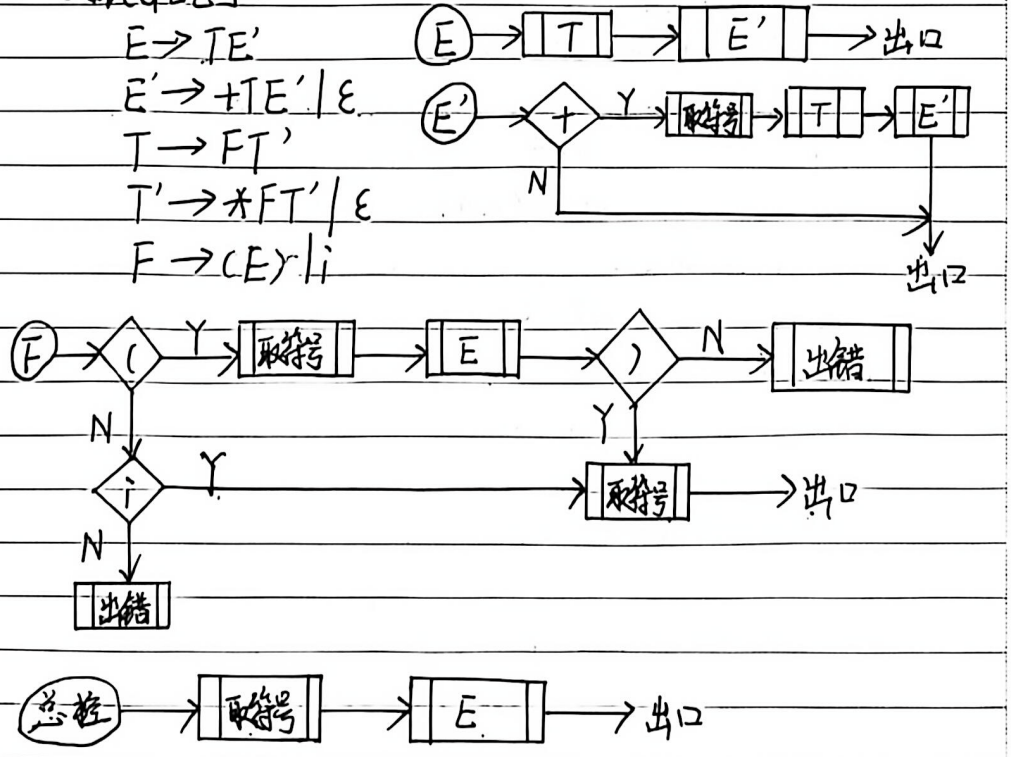
$E \to TE'$

$E' \to +TE' \mid \varepsilon$

$T \to FT'$

$T' \to *FT' \mid \varepsilon$

$F \to (E) \mid i$



Date. /

C语言

```
void GetSymbol(){...}
void Error(){...}
void E()
 { T(); E1(); }
void E1(){     if(sym=='+')
                 {GetSymbol();
                  T();
                  E1();}
          }

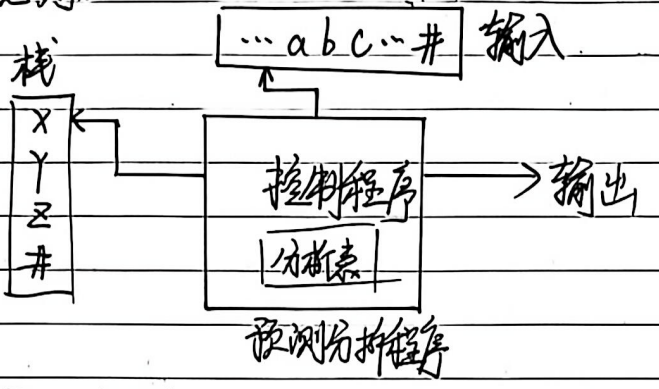
void F()
{   if(sym=='(')
      { GetSymbol();
        E();
        if(sym==')')
          GetSymbol();
        else
          Error();
      }
   else { if(sym=='i')
            GetSymbol();
          else
            Error();}
}
      void main(){ GetSymbol();
                   E();
                 }
```

deli得力

Date. /

表驱动LL(1)分析程序

基本思想：从左到右扫描输入串，同时从识别符号开始生成最左推导，向前看一个符号，以确定当前应选择哪一条规则。



构造预测分析表

步骤1 对文法的每个规则 $U \to u$，执行步骤2与3

步骤2 对于每个终结符 $a \in First(u)$，让

$A[U, a] = 'U \to u'$;

步骤3 如果 $\varepsilon \in First(u)$，则对 $Follow(U)$ 中的每个终结符号 b 或 #，让 $A[U, b] = 'U \to u'$ 或

$A[U, \#] = 'U \to u'$;

步骤4 把A的每个未定义元素置为 ERROR（用空白表示）

输入符号串：i+i*i

是否是文法G[E]的句子。

Date.

设有文法G[E]:

$E ::= TE'$ $E' ::= +TE' | \varepsilon$

$T ::= FT'$ $T' ::= *FT' | \varepsilon$ $F ::= (E) | i$

First(TE') = {(, i}

First(+TE') = {+}

First(ε) = {ε} Follow(E') = {), #}

First(FT') = {(, i}

First(*FT') = {*}

First(ε) = {ε} Follow(T') = {+,), #}

First((E)) = {(}

First(i) = {i}

栈顶 \ 输入	i	+	*	(	)	#
E	E→TE'			E→TE'		
E'		E'→+TE'			E'→ε	E'→ε
T	T→FT'			T→FT'		
T'		T'→ε	T'→*FT'		T'→ε	T'→ε
F	F→i			F→(E)		

栈	输入	输出
#E	i+i*i#	
#E'T	i+i*i#	E→TE'
#E'T'F	i+i*i#	T→FT'
#E'T'i	i+i*i#	F→i
#E'T'	+i*i#	
#E'	+i*i#	T'→ε
#E'T+	+i*i#	E'→+TE'
#E'T	i*i#	
#E'T'F	i*i#	T→FT'
#E'T'i	i*i#	F→i
#E'T'	*i#	
#E'T'F*	*i#	T'→*FT'
#E'T'F	i#	
#E'T'i	i#	F→i
#E'T'	#	
#E'	#	T'→ε

deli得力

Date. /

算符优先分析

自底向上分析
- 优先分析
  - 简单算符 ✓
  - 算符
- LR分析 ✓
  - LR(0)
  - SLR(1)
  - LR(1)

算符优先分析的基本思想则是只规定算符之间的优先关系，也就是只考虑终结符之间的优先关系。

G[E]: E→T|E+T|E-T
T→F|T*F|T/F
F→(E)|i

设有文法G，如果G中没有形如 A→…BC… 的产生式，其中B和C为非终结符，则称G为算符文法。

设G是一个不含ε产生式的算符文法，a和b是任意两个终结符，A、B、C是非终结符，算符优先关系 =、<·、·> 定义如下

(1) a=b 当且仅当G中含有形如 A→…ab… 或A→…aBb… 的产生式

(2) a<·b 当且仅当G中含有形如 A→…aB… 的产生式，且 B⇒b… 或 B⇒Cb…

(3) a·>b 当且仅当G中含有形如 A→…Bb… 的产生式，且 B⇒…a 或 B⇒…aC

(b,a)

设有一个不含ε产生式的算符文法G，如果任一终结符对(a,b)之间至多只有<·、·>和=3种关系中的一种成立，则称G是一个算符优先文法

[强调：两个终结符之间的优先关系是有序的，允许有a·>b，b<·a同时存在，而不允许有a·>b，a<·b，a=b 3种情况中的两种同时存在。]

FIRSTVT(B) = {b | B ⇒+ b… 或 B ⇒+ Cb…}，其中⇒+表示
LASTVT(B) = {a | B ⇒+ …a 或 B ⇒+ …aC} V*的符号串

构造规则

F (1) 若有 T→a… 或 T→Ra…，则 a ∈ FIRSTVT(T)
(2) 若有 a ∈ FIRSTVT(R)，且有产生式 T→R…，则 a ∈ FIRSTVT(T)

L (1) 若有 T→…a 或 T→…aR，则 a ∈ LASTVT(T)
(2) 若 a ∈ LASTVT(R)，且有产生式 T→…R，则 a ∈ LASTVT(T)

对推导的遍历转换成对产生式的反复遍历

例 G[E]: E → E+T | T
T → T*F | F
F → (E) | i

算符优先文法

	+	*	(	)	i
E	✓	✓	✓		✓
T		✓	✓		✓
F			✓		✓

$V_T V_N$ ↳ FIRSTVT
+T
*F
(E

	+	*	(	)	i
E	✓	✓		✓	✓
T		✓		✓	✓
F				✓	✓

$V_N V_T$ LASTVT ↘
E+
T*
E)

	+	*	(	)	i
+	>	<	<	>	<
*	>	>	<	>	<
(	<	<	<	=	<
)	>	>		>	
i	>	>		>	

构造算符优先关系表

deli得力

Date:

素短语是满足下面条件的短语：

1. 至少包含一个终结符
2. 该短语不再包含满足第一个条件的更小的短语

最左边的素短语称为最左素短语

例 已知文法 G[S] 为

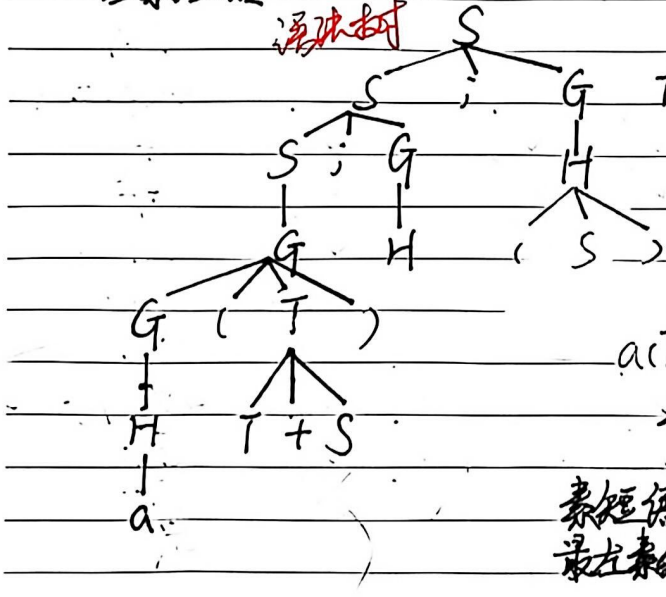
$S \to S;G \mid G$

$G \to G(T) \mid H$

$H \to a \mid (S)$

$T \to T+S \mid S$

给出句型 $a(T+S);H;(S)$ 的短语、句柄、素短语和最左素短语



a是句型相对于H、G的短语

T+S是句型相对于T的短语

a(T+S)是句型相对于G、S的短语

H是…G的短语

a(T+S);H是…S的短语

(S)是句型相对于H、G的短语

a(T+S);H;(S)是…S的短语

其中a是句型的句柄

素短语：a、T+S、(S)

最左素短语：a

G[S]: S→S;G|G
G→G(T)|H
H→a|(S)
T→T+S|S

给出 a;(a+a) 的算符优先分析过程

	;	(	)	a	+
;	>	<	>	<	>
(	<	<	=	<	<
)	>	>	>		>
a	>	>	>		>
+	<	<	>	<	>

栈K	当前输入符	输入串剩余部分	动作
#	a	;(a+a)#	移进
#a	;	(a+a)#	归约
#T	;	(a+a)#	移进
#T;	(	a+a)#	移进
#T;(	a	+a)#	移进
#T;(a	+	a)#	归约
#T;(T	+	a)#	移进
#T;(T+	a	)#	移进
#T;(T+a	)	#	归约
#T;(T+T	)	#	归约
#T;(T	)	#	移进
#T;(T)	#		归约
#T;T	#		归约
#T	#		接受

Date. /

补：设 $G=(V_T, V_N, P, S)$ 是上下文无关文法

$FIRST(\alpha)=\{a \mid \alpha \overset{*}{\Rightarrow} a\beta, a\in V_T, \alpha,\beta\in V^*\}$

若 $\alpha \overset{*}{\Rightarrow} \varepsilon$，则规定 $\varepsilon \in FIRST(\alpha)$。

称 $FIRST(\alpha)$ 为 α 的首符号集。

$A \to a \mid b \mid \varepsilon$

$B \to c \mid d$

$C \to e \mid f \mid \varepsilon$

$FIRST(A)=\{a, b, \varepsilon\}$

$FIRST(AB)=\{a, b, c, d\}$

$FIRST(AC)=\{a, b, e, f, \varepsilon\}$

$FIRST(ACf)=\{a, b, e, f\}$

$FIRST(ACB)=\{a, b, e, f, c, d\}$

$FIRST(ABC)=\{a, b, c, d\}$

$FIRST(BAC)=\{c, d\}$

LR分析（自底向上语法分析）

1. LR(0)项目集规范族的构造

构造识别活前缀的DFA（框图）P133

构成识别一个文法活前缀的DFA项目集（状态）的全体称为这个文法的LR(0)项目集规范族（列表）P138

LR(0)项目：如果 $U::=uv$ 是文法G的一个规则，其中u或v可为空串，则 $U \to u\cdot v$ 称为G的一个LR(0)项目

（在文法G中每个产生式的右部适当位置添加一个圆点构成项目）

例：产生式 $S \to aAcBe$ 对应有6个项目

$S \to \cdot aAcBe$，$S \to a\cdot AcBe$，$S \to aA\cdot cBe$

$S \to aAc \cdot Be$ 、$S \to aAcB \cdot e$、$S \to aAcBe \cdot$

*对于空产生式，$A \to \varepsilon$ 仅有一个项目 $A \to \cdot$

每个项目的含义与圆点的位置有关

移进项：圆点后面为终结符的项目

待约项：圆点后面为非终结符的项目

归约项：圆点在最右端的项目

特殊 → 接受项：如果一个项呈 $Z \to u \cdot$ 形，Z是识别符号

G[Z]：$Z' \to Z$ 也行

初始项：文法G[Z]的LR(0)项 $Z \to \cdot u$ 称为G的初始项

后继项：设 $V \to u \cdot Av$ 是文法G的一个LR(0)项，其中 $A \in V$，则LR(0)项 $V \to uA \cdot v$ 称为它的后继项。（圆点向右移动）

项目

项集：由LR(0)项组成的集合称LR(0)项集

后继项集：当识别程序所处状态所对应的项集中有一个项，其中圆点后面是符号X，则识别程序在该符号X下将进入的处状态的X后继状态。相应的项集称X后继项集。

每个项集S_i的后继项集S通常是基本项集的闭包集合，基本项集可直接由项集S_i生成

即 $\{V \to uA \cdot v \mid (V \to u \cdot Av) \in S_i\}$

项集的闭包：设S_i是文法G的一个项集，项集S_j的闭包CLOSURE(S_j)是按下列步骤构造而得的项集

步骤1 S_j中每个项在CLOSURE中

步骤2 如果 $V \to u \cdot Vv \in CLOSURE(S_j)$，且 $V ::= w$ 是一个规则，则把 $V \to \cdot w$ 添入CLOSURE(S_j)中

步骤3 重复步骤2，直到CLOSURE(S_j)不再扩大

deli得力

Date. /

继承属性：出现在规则右部的符号的属性。一个结点相应文法符号的属性之值通过语法树中它的兄弟结点、与父结点相应文法符号的属性之值来计算。（自顶向下）

例：

产生式	语义规则
$L ::= E$	{ print(E.val) } 副作用
$E ::= E_1 + T$	{ E.val := E_1.val + T.val }
$E ::= T$	{ E.val := T.val }
$T ::= T_1 * F$	{ T.val := T_1.val * F.val }
$T ::= F$	{ T.val := F.val }
$F ::= (E)$	{ F.val := E.val }
$F ::= digit$	{ F.val := digit.val } 内在属性

产生式	语义规则
$D \rightarrow TL$	{ L.in := T.type }
$T \rightarrow int$	{ T.type := integer }
$T \rightarrow float$	{ T.type := real }
$L \rightarrow L_1, id$	{ L_1.in := L.in 副作用 addtype(id.entry, L.in) 内在属性 }
$L \rightarrow id$	{ addtype(id.entry, L.in) }

内在属性：不会出现在属性文法出现集合中的属性，通常是终结符的属性（终结符的综合属性）。如 digit.val 是由词法分析程序提供的属性值

* 有时语义规则的目的是为了产生一个副作用，比如打印计算结果或添加符号表。此时，语义规则通常被写成过程调用的形式，或者作为产生式左部文法符号的虚综合属性的定义。

$f(c_1, c_2, \cdots, c_k)$

OCR得力

Dole. /

一个没有副作用的SDD，语法制导定义，有时称为属性文法。（语义规则仅通过其结点属性值和常量来定义计属性值）
SDD是一个上下文无关文法、属性和语义规则的结合。

△遍历语法树进行语义计算
依赖图
拓扑排序：1. 从图中选择一个入度为0的结点，（有向无环图）并计算其属性值
2. 删除该结点，以及以它为起点的有向边
3. 重复1.2，直到图为空
带标注的语法分析树

S-属性文法 基于∧的语义计算（自底向上）
L-属性文法
基于L-属性文法的语义计算
△树遍历 深度优先从左至右遍历分析树（function visit）
后根遍历（N叉树）
假设语法树已建立，树中已带有开始符号的继承属性和终结符的综合属性
算法如下
While 还有未被计算的属性 do
Visit Node(S) /*S是开始符号*/

procedure Visit Node (N: node);
begin

Date

if N是一个非终结符 then //*假设其产生式为N→$X_1 \cdots X_m$*/

for i:=1 to m do

if $X_i \in V_N$ then //*即X_i为非终结符*/

begin

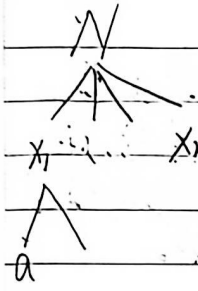
计算X_i的所有能够计算的继承属性;

VisitNode(X_i)

end;

计算N的所有能够计算的综合属性

end



S-翻译模式

L-翻译模式

基于L-翻译模式的自顶向下语义计算

以继承属性为形参 以综合属性为返回值

源程序的中间表示

一、抽象语法树(AST)

对输入串按照翻译模式进行语义计算

1°若该SDT的基础文法是二义性的

则构造LR分析表,进行自底向上的句型分析

(归约时进行属性的求值)

2°若该SDT的基础文法是无二义性的

同理,可使用LR分析,也可以遍历语法分析树

进行属性值的计算

*(这里默认为S-翻译模式。如果遇到L-属性文法或翻译模式,需要采用自顶向下的分析)

deli得力

Date. /

构造AST的一种技巧

(1) 从当前的中缀表达式中选择最后执行的四则运算符，作为当前的根节点，若没有运算符，将数字写在对应的节点处。

(2) 该运算符将原表达式分为左右子树，重复步骤(1)

(注意：在数学中，对于优先级相等的运算符，通常是从左向右计算。这与左递归的表达式文法一致。但右递归的表达式文法是从右向左计算的。)

二、逆波兰表示（后缀表达式） $a+b \leftrightarrow ab+$

1. 对表达式构造AST，后根遍历得到
2. 按照相应的翻译模式进行语义计算
3. 将中缀表达式转为后缀式的算法

分配两个栈，一个临时存储运算符的栈S1和一个存放结果的栈S2。从中缀式的左端依次读入字符。

(1) 若取出操作数，则压入栈S2

(2) 若取出运算符，则将该运算符与S1栈顶元素比较，如果优先级高于，则将该运算符压入S1，否则，将栈顶元素出栈并压入S2，直至栈顶元素优先级低于该运算符，最后将该运算符压入S1

(3) 若取出"("，则压入S1

(4) 若取出")"，则将S1中的运算符依次出栈，并压入S2，直到遇到"("为止，此时抛弃"("

直到遍历完整个中缀式之后，若S1中仍存在运算符，则全部依次出栈并压入S2。

完成上述步骤后，将S2逆序输出，得到后缀式。

4. 中缀式转为后缀式的一种技巧（类似BST）

(1) 选择当前中缀式中最后执行的运算符，将其置于后缀式的尾部。此时，中缀式会分为左右两个较小的，重复上述步骤。

仅供参考

(2) 也可选择最先执行的运算符，得到一个基础的后缀式。重复上述步骤，不断的写该后缀式，直至所有运算符都入式。

补. 后缀式转化为中缀式的方法

从左向右扫描后缀式，遇到运算符，就将其与前面两个操作数结合，在原式中作为新的操作数。同时得到一个基础的中缀式。重复，扫写，直至。最后，去除多余括号，得中缀式。 ↑加上括号

三、四元式序列（一个[illegible]的机制[指令翻译]）

（运算符，运算分量，运算分量，结果）

其中，运算分量和结果可以是变量、常量或临时变量

Date. /

状态	项目集	后继符号	后继状态
S_1	$\{S'\to S\cdot\}$	#$S'\to S$	S_{11}
S_2	$\{S\to A\cdot\}$	#$S\to A$	S_{11}
S_3	$\{S\to B\cdot\}$	#$S\to B$	S_{11}
S_4	$\{A\to a\cdot Ab$	A	S_7
	$B\to a\cdot Bb$	B	S_8
	$A\to\cdot aAb$	a	S_4
	$B\to\cdot aBb$	a	S_4
	$A\to\cdot c$	c	S_5
	$B\to\cdot d\}$	d	S_6
S_5	$\{A\to c\cdot\}$	#$A\to c$	S_{11}
S_6	$\{B\to d\cdot\}$	#$B\to d$	S_{11}
S_7	$\{A\to aA\cdot b\}$	b	S_9
S_8	$\{B\to aB\cdot b\}$	b	S_{10}
S_9	$\{A\to aAb\cdot\}$	#$A\to aAb$	S_{11}
S_{10}	$\{B\to aBb\cdot\}$	#$B\to aBb$	S_{11}

项集规范族和特征有穷状态机

文法的LR(0)项集规范族可以被抽象成一个有穷状态机(FSM)

其步骤如下：

步骤1 把各个项集定义为该FSM的内部状态，并用项集的编号来命名这个状态。因此，每个项集在FSM中有一个对应状态。

步骤2 让该FSM状态之间的转换对应于后继关系。

步骤3 与初始项集对应的状态作为该FSM的开始状态，与归约-后继项集对应的状态作为该FSM的终止状态

这种有穷状态机FSM称为文法的特征有穷状态机

LR(0)文法的定义(两个) GO(I,X)转换函数

LR(0)分析表的构造

对输入串的LR(0)分析过程

SLR(1)分析

简单向前看1条符号：一类是文法符号X下的转换(称X_转换)，集合便是{X}（移进项）

另一类是#U::=u转换，集合是Follow(U)（归约项） P139

SLR(1)文法的判定 { 项集中的冲突都能用SLR(1)的方法解决 / SLR(1)分析表中无重复定义

SLR(1)分析表的构造 P142

SLR(1)分析过程

LR(1)分析

LR(1)项目、LR(1)项集的闭包、LR(1)项集规范族的构造

LR(1)分析表的构造

例：设有文法G[S']：(0) S'→S (4) B→C

(1) S→CbBA (5) B→Db

(2) A→Aab (6) C→a

(3) A→ab (7) D→a

解：其LR(1)项集规范族如下

状态	LR(1)项集	后继符号	后继状态
S_0	{[S'→·S,#]	S	S_1
	[S→·CbBA,#]	C	S_2
	[C→·a,b]}	a	S_3

Date.

状态	项目	后继符号	后继状态
S_1	$\{[S'\to S\cdot,\#]\}$	$\#S'\to S$	S_{15}
S_2	$\{[S\to C\cdot bBA,\#]\}$	b	S_4
S_3	$\{[C\to a\cdot,b]\}$	$\#C\to a$	S_{15}
S_4	$\{[S\to Cb\cdot BA,\#]$	B	S_5
	$[B\to\cdot C,a]$	C	S_6
	$[B\to\cdot Db,a]$	D	S_7
	$[C\to\cdot a,a]$	a	$\}S_8$
	$[D\to\cdot a,b]\}$	a	
S_5	$\{[S\to CbB\cdot A,\#]$	A	$\}S_9$
	$[A\to\cdot Aab,\#\|a]$	A	
	$[A\to\cdot ab,\#\|a]\}$	a	S_{10}
S_6	$\{[B\to C\cdot,a]\}$	$\#B\to C$	S_{15}
S_7	$\{[B\to D\cdot b,a]\}$	b	S_{11}
S_8	$\{[C\to a\cdot,a]$	$\#C\to a$	S_{15}
	$[D\to a\cdot,b]\}$	$\#D\to a$	S_{15}
S_9	$\{[S\to CbBA\cdot,\#]$	$\#S\to CbBA$	S_{15}
	$[A\to A\cdot ab,\#\|a]\}$	a	S_{12}
S_{10}	$\{[A\to a\cdot b,\#\|a]\}$	b	S_{13}
S_{11}	$\{[B\to Db\cdot,a]\}$	$\#B\to Db$	S_{15}
S_{12}	$\{[A\to Aa\cdot b,\#\|a]\}$	b	S_{14}
S_{13}	$\{[A\to ab\cdot,\#\|a]\}$	$\#A\to ab$	S_{15}
S_{14}	$\{[A\to Aab\cdot,\#\|a]\}$	$\#A\to Aab$	S_{15}

deli得力

Date. /

其LR(1)分析表如下

状态	Action				Goto				
	a	b	c	#	S	A	B	C	D
0	S_3				1			2	
1				acc					
2		S_4							
3		r_6							
4	S_8						5	6	7
5	S_{10}				9				
6	r_4								
7		S_{11}							
8	r_6	r_7							
9	S_{12}			r_1					
10		S_{13}							
11	r_5								
12		S_{14}							
13	r_3			r_3					
14	r_2			r_2					

语法制导翻译(Syntax-Directed Translation) P160

龙：编译器设计的复杂性

剑：LALR语法分析 盾：↓

属性、属性文法、语义规则 { 语义动作 P161
条件谓词

综合属性：规则左部符号的属性是综合属性。一个结点相应文法符号的属性之值通过语法分析树中它的子结点的属性之值来计算。(自底向上)